

Final Project

Amrita Acharya & Isabella Lazarov

2026-04-15

Import data and libraries

```
#####  
# UPLOAD LIBRARIES AND FUNCTIONS #  
#####  
  
library(tidyverse)  
library(GenomicRanges)  
library(TxDb.Hsapiens.UCSC.hg38.knownGene)  
library(AnnotationDbi)  
library(org.Hs.eg.db)  
library(glmnet)  
library(depmixS4)  
library(here)  
source(here("fit_hmm_per_gene.R"))  
source(here("fit_lasso_per_gene.R"))  
  
select <- dplyr::select  
slice <- dplyr::slice
```

```
#####  
# UPLOAD DATA #  
#####  
  
if (!use_sample) {  
  
  methyl_raw <- read_tsv("data/methylation/methylation_data.tsv.gz")  
  colnames(methyl_raw)[1] <- "probe_id" #CpG site ids  
  probe_map <- read_tsv("data/methylation/probe_map")  
  colnames(probe_map)[colnames(probe_map) == "#id"] <- "id"  
  
  rna_raw <- read_tsv("data/gene_expression/gene_expression_data.tsv.gz")  
  colnames(rna_raw)[1] <- "gene_id"  
  
  phenotype <- read_tsv("data/phenotype_data.tsv.gz")  
  
  hg38_txdb <- TxDb.Hsapiens.UCSC.hg38.knownGene  
  
}
```

```
#####
# FILTER SAMPLES #
#####
if (!use_sample) {

  # filter for primary tumor samples to reduce noise
  samples <- filter(phenotype, sample_type.samples == "Primary Tumor")
  samples <- pull(samples, sample)

  # samples in all three datasets:
  # methyl_matrix, rna_std, phenotype
  matched_samples <- Reduce(
    intersect, list(colnames(methyl_raw)[-1], colnames(rna_raw)[-1], samples))

  # modify methyl and rna matrices to be matched subset
  methyl_matrix <- select(methyl_raw, probe_id, all_of(matched_samples))
  rna_matrix <- select(rna_raw, gene_id, all_of(matched_samples))

}
```

Standardize RNA-seq IDs

```
#####
# STANDARDIZE IDS #
#####

if (!use_sample) {

  # remove version number from ids
  rna_with_ver <- mutate(rna_matrix, gene_id_no_ver = sub("\\..*$", "", gene_id))

  # change ids from ENSEMBL to ENTREZID
  ensg_keys <- unique(rna_with_ver$gene_id_no_ver)
  ensg_to_entrez <- AnnotationDbi::mapIds(
    org.Hs.eg.db,
    keys = ensg_keys,
    keytype = "ENSEMBL",
    column = "ENTREZID",
    multiVals = "first"
  )

  # final matched rna dataset
  rna_std <- mutate(
    rna_with_ver, gene_id_entrez = as.character(ensg_to_entrez[gene_id_no_ver])
  )
  rna_std <- filter(rna_std, !is.na(gene_id_entrez))
  rna_std <- group_by(rna_std, gene_id_entrez)
  rna_std <- summarize(
    rna_std, across(all_of(matched_samples), mean, na.rm = TRUE))

}
```

Extract promoter regions

Methylation

```
#####  
# FILTER METHYLATION DATA FOR PROMOTER REGIONS #  
#####  
  
if (!use_sample) {  
  
  # make GRanges object, assign probe ID to chromosome ranges  
  methyl_gr <- GRanges(  
    seqnames = probe_map$chrom,  
    ranges = IRanges(start = probe_map$chromStart, end = probe_map$chromEnd),  
    probe_id = probe_map$id)  
  
  # extract promoter regions  
  promoter_regions <- genes(hg38_txdb, single.strand.genes.only=FALSE)  
  promoter_regions <- promoters(promoter_regions, upstream=2000, downstream=400)  
  
  # find overlaps of CpGs in methylation data and gene promoter regions  
  probe_overlaps <- overlapsAny(methyl_gr, promoter_regions, ignore.strand = TRUE)  
  probes_in_promoter <- probe_map$id[probe_overlaps]  
  
  # filter out CpGs not in promoter region from methyl_matrix  
  methyl_filtered <- filter(methyl_matrix, probe_id %in% probes_in_promoter)  
  
}
```

RNA-seq

```
#####  
# FILTER RNA DATA FOR PROMOTER REGIONS #  
#####  
  
if (!use_sample) {  
  
  hg38_genes <- genes(hg38_txdb)  
  genes_in_rna_std <- hg38_genes$gene_id %in% unique(na.omit(rna_std$gene_id_entrez))  
  rna_seq_ranges <- hg38_genes[genes_in_rna_std]  
  rna_seq_promoters <- promoters(rna_seq_ranges, upstream=2000, downstream=400)  
  
}
```

```
#####  
# MERGE METHYLATION AND RNA DATA #  
#####  
  
if (!use_sample) {
```

```

# filter out methylation data not in promoter regions
methy1_gr_filtered <- methy1_gr[mcols(methy1_gr)$probe_id %in% probes_in_promoter]
methy1_rna_overlaps <- findOverlaps(
  rna_seq_promoters, methy1_gr_filtered, ignore.strand = TRUE)

# dataframe of matched CpG and gene ids
probe_gene_map <- data.frame(
  probe_id = mcols(
    methy1_gr_filtered)$probe_id[subjectHits(methy1_rna_overlaps)],
  gene_id_entrez = mcols(
    rna_seq_promoters)$gene_id[queryHits(methy1_rna_overlaps)],
  chrom = as.character(
    seqnames(methy1_gr_filtered)[subjectHits(methy1_rna_overlaps)],
    start = start(methy1_gr_filtered)[subjectHits(methy1_rna_overlaps)])

#pivot data long: one row per CpG-sample combo for HMM
methy1_long_format <- pivot_longer(
  methy1_filtered,
  cols = -probe_id,
  names_to = "sample_id",
  values_to = "beta")
methy1_long_format <- inner_join(
  methy1_long_format, probe_gene_map, by = "probe_id")

#pivot data long: one row per gene-sample combo for HMM
rna_long_format <- pivot_longer(
  rna_std,
  cols = -gene_id_entrez,
  names_to = "sample_id",
  values_to = "expression")

# merge two long formatted methyl and rna datasets
merged_df <- inner_join(
  methy1_long_format, rna_long_format, by = c("sample_id", "gene_id_entrez"))

head(merged_df)
}

```

Analyses

```

#####
# CREATE SMALL SAMPLE FILE #
#####

# later filtering of genes with fewer CpG sites for HMM may produce worse results
sample_genes <- head(unique(merged_df$gene_id_entrez), 20)
sample_input_data <- filter(merged_df, gene_id_entrez %in% sample_genes)
write_csv(sample_input_data, "sample_input.csv")

```

```
#####
# LOAD SMALL SAMPLE FILE #
#####
```

```
if (use_sample) {

  # first 20 genes in order of larger dataset
  merged_df <- read_csv("sample_input.csv")

}
```

```
## Rows: 66708 Columns: 7
## -- Column specification -----
## Delimiter: ","
## chr (3): probe_id, sample_id, chrom
## dbl (4): beta, gene_id_entrez, start, expression
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

```
#####
# LASSO ANALYSIS #
#####
```

```
set.seed(123)

cpgs_per_gene <- summarize(group_by(merged_df, gene_id_entrez), n_cpgs = n_distinct(probe_id))

# get 1000 genes with most CpG sites
target_genes <- arrange(cpgs_per_gene, desc(n_cpgs))
target_genes <- slice_head(target_genes, n = 1000)
target_genes <- pull(target_genes, gene_id_entrez)

lasso_result <- NULL
lasso_result <- data.frame()
all_fits <- list()
for (gene in target_genes) {
  result <- fit_lasso_per_gene(gene, merged_df)
  if (!is.null(result)) {
    lasso_result <- bind_rows(lasso_result, result$summary)
    all_fits[[as.character(gene)]] <- result$cv_fit
  }
}
```

```
## Warning in cor(Y_test, predicted_expression): the standard deviation is zero
```

```
lasso_result
```

```
##   gene_id_entrez n_cpgs n_samples   r_squared
## 1          6422     19         299 0.389045059
## 2          57178     15         299 0.003115792
## 3          10421     14         305 0.401798662
```

```
## 4      81614      14      304 0.012236279
## 5      283050     10      295      NA
## 6       7555       8      306 0.021631731
## 7      10294       6      306 0.131937493
## 8       5934       5      304 0.057365192
```

```
# better R^2 results than Zhong et al.,
# but we run lasso for 1000 genes as opposed to their 4000+
paste0("Percentage of genes with R^2 > 0.3 = ", mean(lasso_result$r_squared > 0.3, na.rm = TRUE)*100)
```

```
## [1] "Percentage of genes with R^2 > 0.3 = 28.5714285714286"
```

```
paste0("Percentage of genes with R^2 > 0.5 = ", mean(lasso_result$r_squared > 0.5, na.rm = TRUE)*100)
```

```
## [1] "Percentage of genes with R^2 > 0.5 = 0"
```

```
paste0("Percentage of genes with R^2 > 0.8 = ", mean(lasso_result$r_squared > 0.8, na.rm = TRUE)*100)
```

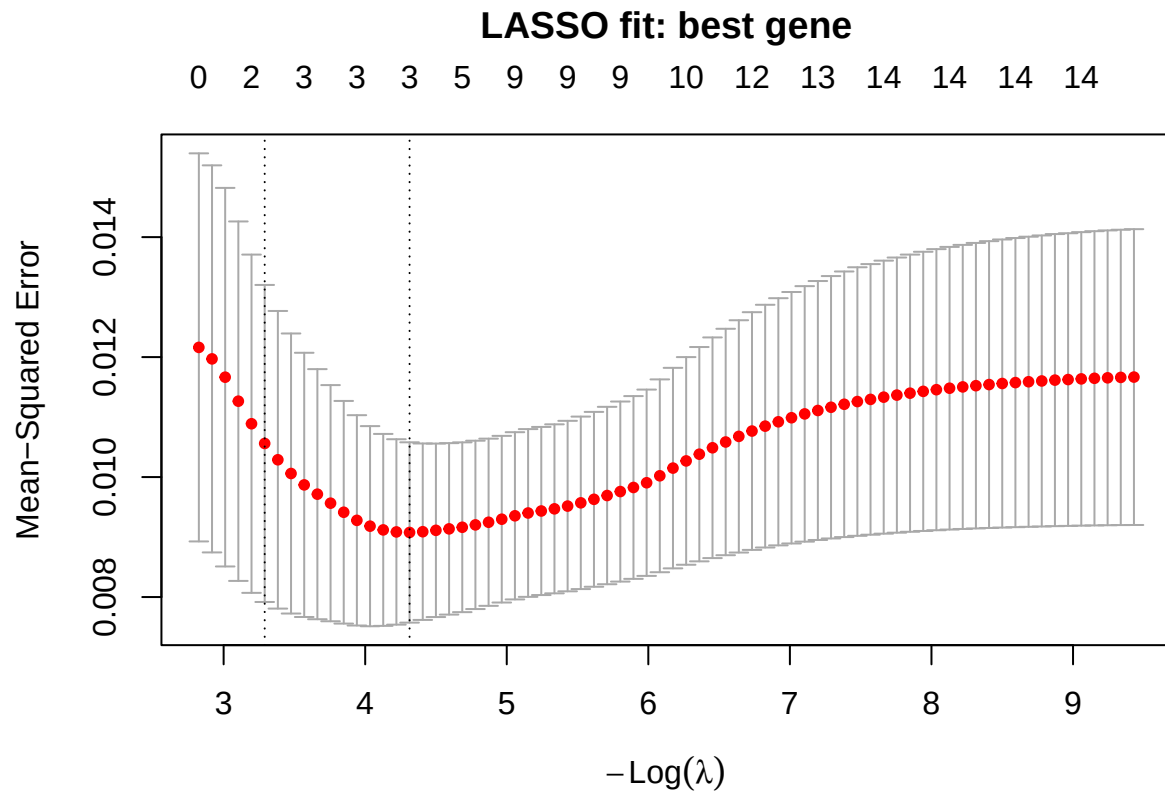
```
## [1] "Percentage of genes with R^2 > 0.8 = 0"
```

```
#####
# LASSO SIGNAL VISUALS: WORST AND BEST GENES #
#####

# signal visualization of worst (lowest R^2)...
lasso_gene_rankings_asc <- arrange(lasso_result, r_squared)
worst_gene_lasso <- slice(lasso_gene_rankings_asc, 1)
worst_gene_lasso_id <- pull(worst_gene_lasso, gene_id_entrez)
worst_gene_r_squared <- pull(worst_gene_lasso, r_squared)

# ... and best (highest R^2) genes
lasso_gene_rankings_desc <- arrange(lasso_result, desc(r_squared))
best_gene_lasso <- slice(lasso_gene_rankings_desc, 1)
best_gene_lasso_id <- pull(best_gene_lasso, gene_id_entrez)
best_gene_lasso_r_squared <- pull(best_gene_lasso, r_squared)

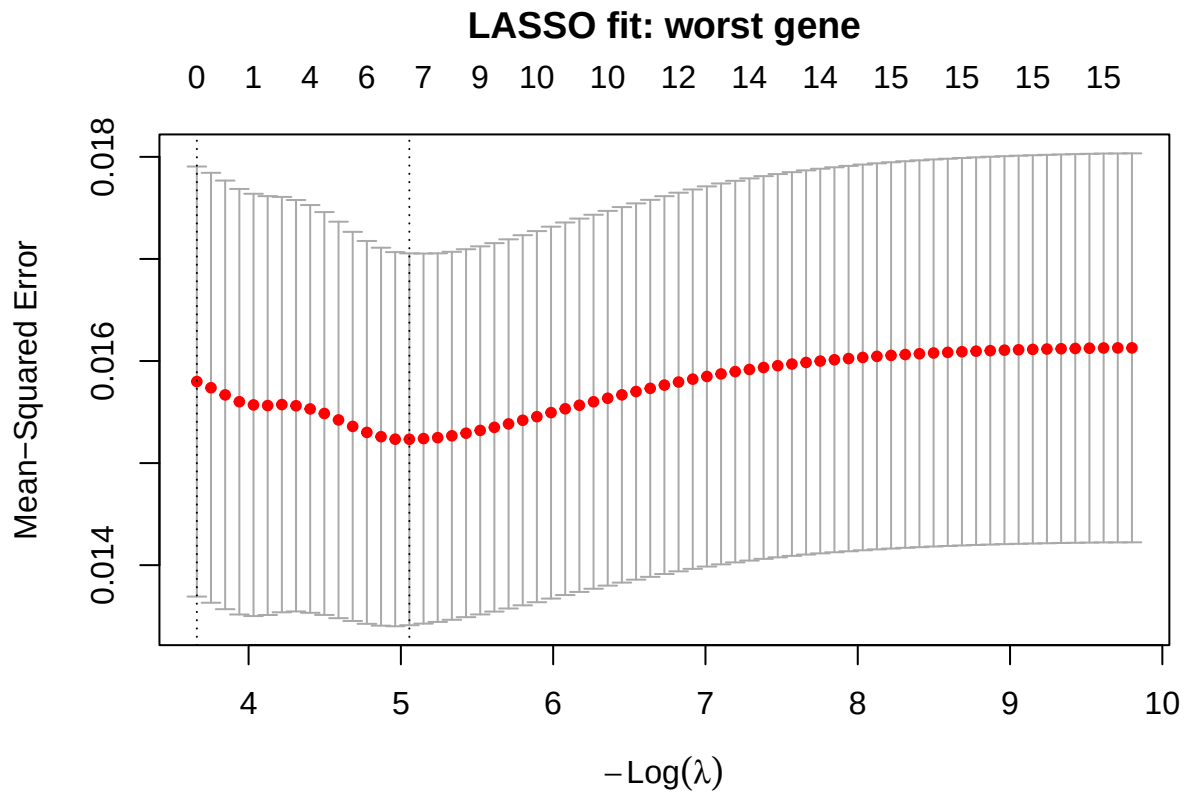
best_fit <- all_fits[[as.character(best_gene_lasso_id)]]
plot(best_fit)
title("LASSO fit: best gene", line = 2.5)
```



```
paste0("best gene r^2 = ", best_gene_lasso_r_squared)
```

```
## [1] "best gene r^2 = 0.4017986615648"
```

```
worst_fit <- all_fits[[as.character(worst_gene_lasso_id)]]
plot(worst_fit)
title("LASSO fit: worst gene", line = 2.5)
```



```
paste0("worst gene r^2 = ", worst_gene_r_squared)
```

```
## [1] "worst gene r^2 = 0.00311579182310775"
```

```
#####
# Correlation analysis + plot #
#####
# averaging all methylation values of a single gene to determine if there is
# a simple correlation between methylation and gene expression

# average methylation per gene
gene_summary <- summarize(
  group_by(merged_df, gene_id_entrez),
  avg_methylation = mean(beta, na.rm = TRUE),
  avg_expression = mean(expression, na.rm = TRUE)
)

# correlation using all genes including those w/o expression
# some may be silent due to reasons other than methylation
cor_all <- cor(
  gene_summary$avg_methylation, gene_summary$avg_expression, use = "complete.obs")
paste("correlation (all genes) = ", round(cor_all, 4))
```

```
## [1] "correlation (all genes) = -0.4455"
```

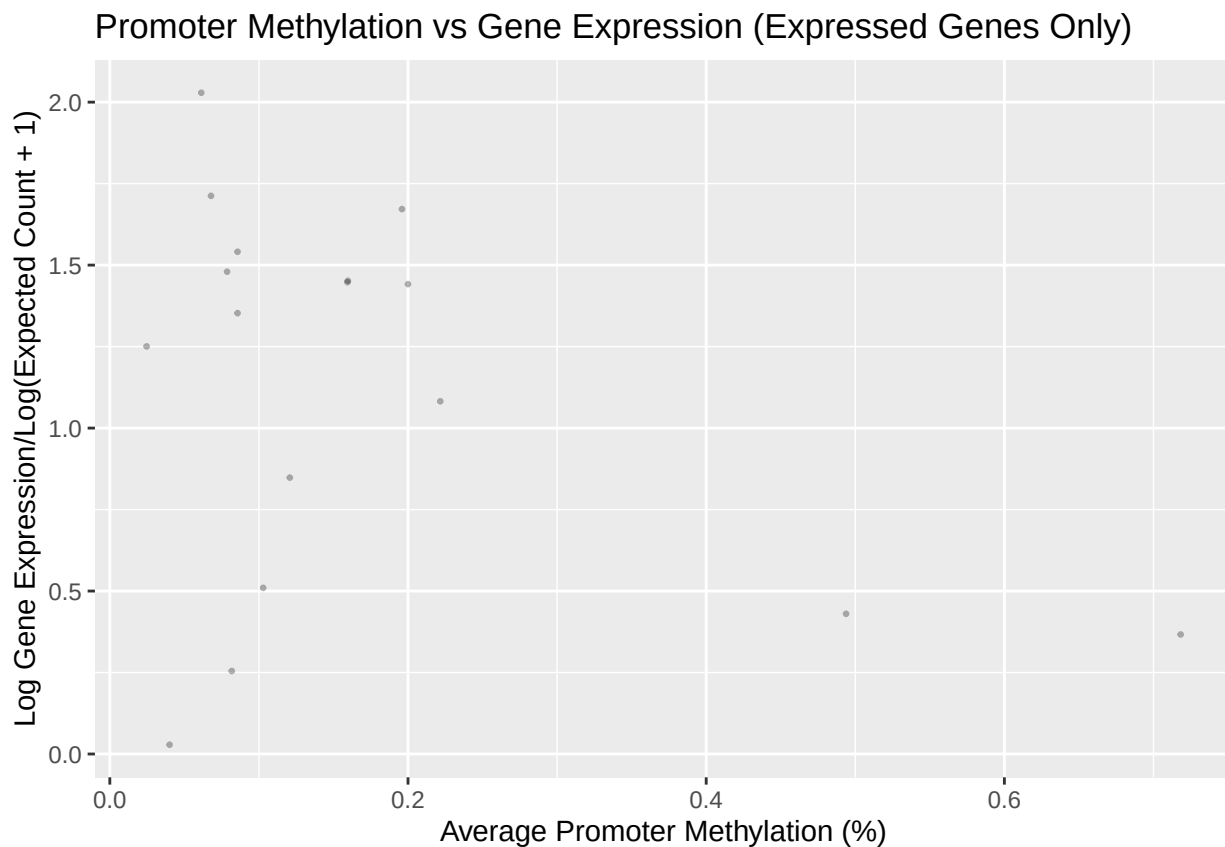


```
# correlation using only genes with expression > 0
gene_summary_expressed <- filter(gene_summary, avg_expression > 0)
cor_expressed <- cor(gene_summary_expressed$avg_methylation,
                     gene_summary_expressed$avg_expression,
                     use = "complete.obs")
paste("correlation (expressed genes only) = ", round(cor_expressed, 4))
```

```
## [1] "correlation (expressed genes only) = -0.3697"
```

```
ggplot(
  gene_summary_expressed,
  aes(x = avg_methylation, y = log1p(avg_expression))) +
  geom_point(alpha = 0.3, size = 0.5) +
  labs(x = "Average Promoter Methylation (%)",
       y = "Log Gene Expression/Log(Expected Count + 1)",
       title = "Promoter Methylation vs Gene Expression (Expressed Genes Only)")
```

```
## Warning: Removed 1 row containing missing values or values outside the scale range
## ('geom_point()').
```



```
#####
# HMM Analysis #
#####
```

```

# same genes as LASSO analysis for reliable comparison
hmm_target_genes <- lasso_result$gene_id_entrez

# sort CpG sites by position in genome so that they can be modeled as sequence
hmm_data <- arrange(merged_df, gene_id_entrez, sample_id, start)

# log-transform gene expression values
hmm_data$log_expression <- log1p(hmm_data$expression)

hmm_result <- NULL
hmm_result <- data.frame()
all_hmm_fits <- list()
all_hmm_models <- list()
for (gene in hmm_target_genes) {
  result <- fit_hmm_per_gene(gene, hmm_data)
  if (!is.null(result)) {
    hmm_result <- bind_rows(hmm_result, result$summary)
    all_hmm_fits[[as.character(gene)]] <- result$lm_fit
    all_hmm_models[[as.character(gene)]] <- result$hmm_fit
  }
}

```

```

## converged at iteration 31 with logLik: -2154.033
## converged at iteration 102 with logLik: 1556.317
## converged at iteration 59 with logLik: 1803.78
## converged at iteration 38 with logLik: 318.5625
## converged at iteration 210 with logLik: 1092.464

```

```
hmm_result
```

```

##   gene_id_entrez num_cpgs n_samples r_squared
## 1           6422      19         306 0.7964954
## 2          57178      15         306 0.7686470
## 3          10421      14         306 0.3492514
## 4          81614      14         306 0.6012640
## 5         283050      10         306 0.6529973

```

```
paste0("Percentage of genes with R^2 > 0.3 = ", mean(hmm_result$r_squared > 0.3, na.rm = TRUE)*100)
```

```
## [1] "Percentage of genes with R^2 > 0.3 = 100"
```

```
paste0("Percentage of genes with R^2 > 0.5 = ", mean(hmm_result$r_squared > 0.5, na.rm = TRUE)*100)
```

```
## [1] "Percentage of genes with R^2 > 0.5 = 80"
```

```
paste0("Percentage of genes with R^2 > 0.8 = ", mean(hmm_result$r_squared > 0.8, na.rm = TRUE)*100)
```

```
## [1] "Percentage of genes with R^2 > 0.8 = 0"
```

```

# n genes = 1000
# "Percentage of genes with R^2 > 0.3 = 67.3366834170854"
# "Percentage of genes with R^2 > 0.5 = 56.5326633165829"
# "Percentage of genes with R^2 > 0.8 = 8.29145728643216"

#####
# EXTRACT TRANSITION PROBABILITIES #
#####

hmm_gene_rankings_desc <- arrange(hmm_result, desc(r_squared))
best_gene_hmm <- slice(hmm_gene_rankings_desc, 1)
best_gene_hmm_id <- pull(best_gene_hmm, gene_id_entrez)
best_gene_hmm_r_squared <- pull(best_gene_hmm, r_squared)

worst_gene_hmm <- filter(hmm_gene_rankings_desc, !is.na(r_squared))
num_genes_fit <- nrow(worst_gene_hmm)
worst_gene_hmm <- slice(worst_gene_hmm, num_genes_fit)
worst_gene_hmm_id <- pull(worst_gene_hmm, gene_id_entrez)
worst_gene_hmm_r_squared <- pull(worst_gene_hmm, r_squared)

best_hmm <- all_hmm_models[[as.character(best_gene_hmm_id)]]
worst_hmm <- all_hmm_models[[as.character(worst_gene_hmm_id)]]

# fit hmm on best and worst genes each
# can change best_hmm and worse_hmm as desired
transition_matrix_best <- matrix(nrow = 3, ncol = 3)
for(i in 1:3) {
  transition_matrix_best[i, ] <- getpars(getmodel(best_hmm, "transition", i))
}

transition_matrix_worse <- matrix(nrow = 3, ncol = 3)
for(i in 1:3) {
  transition_matrix_worse[i, ] <- getpars(getmodel(worst_hmm, "transition", i))
}

paste0("transition probability matrix for best gene:")

## [1] "transition probability matrix for best gene:"

paste0(round(transition_matrix_best, 4))

## [1] "1" "0" "0" "0" "1" "0" "0" "0" "1"

paste0("best gene r^2 = ", best_gene_hmm_r_squared)

## [1] "best gene r^2 = 0.796495419050209"

paste0("transition probability matrix for worst gene:")

## [1] "transition probability matrix for worst gene:"

```

```
paste0(round(transition_matrix_worse, 4))
```

```
## [1] "1" "0" "0" "0" "1" "0" "0" "0" "1"
```

```
paste0("worst gene r^2 = ", worst_gene_hmm_r_squared)
```

```
## [1] "worst gene r^2 = 0.349251400514088"
```

```
#####  
# EXTRACT STATE EMISSIONS #  
#####
```

```
# extract emission parameters for best gene
```

```
state_emissions <- data.frame()
```

```
for (i in 1:3) {
```

```
  methylation_params <- getpars(best_hmm@response[[i]][[1]])
```

```
  expression_params <- getpars(best_hmm@response[[i]][[2]])
```

```
  state_emissions <- rbind(state_emissions, data.frame(  
    state = paste("state", i),
```

```
    methylation_logit_mean = methylation_params[1],
```

```
    methylation_logit_sd = methylation_params[2],
```

```
    expression_intercept = expression_params[1],
```

```
    expression_slope = expression_params[2]
```

```
  ))
```

```
}
```

```
emissions_transformed <- mutate(state_emissions,
```

```
  methylation_beta_mean = plogis(methylation_logit_mean))
```

```
state_emissions <- select(emissions_transformed,
```

```
  state,
```

```
  methylation_beta_mean,
```

```
  methylation_logit_sd,
```

```
  expression_intercept,
```

```
  expression_slope)
```

```
state_emissions <- arrange(state_emissions, state)
```

```
rownames(state_emissions) <- NULL
```

```
print("state emissions for best gene:")
```

```
## [1] "state emissions for best gene:"
```

```
print(state_emissions, digits = 3)
```

```
##      state methylation_beta_mean methylation_logit_sd expression_intercept  
## 1 state 1                0.459                1.077                0.2684  
## 2 state 2                0.511                1.187                0.0563  
## 3 state 3                0.429                0.995                0.9294  
##      expression_slope
```

```
## 1      -0.00535
## 2      -0.00144
## 3      -0.04305
```

```
#####
# EMISSIONS REGRESSION PLOT #
#####
```

```
# evenly spaced out x-axis
```

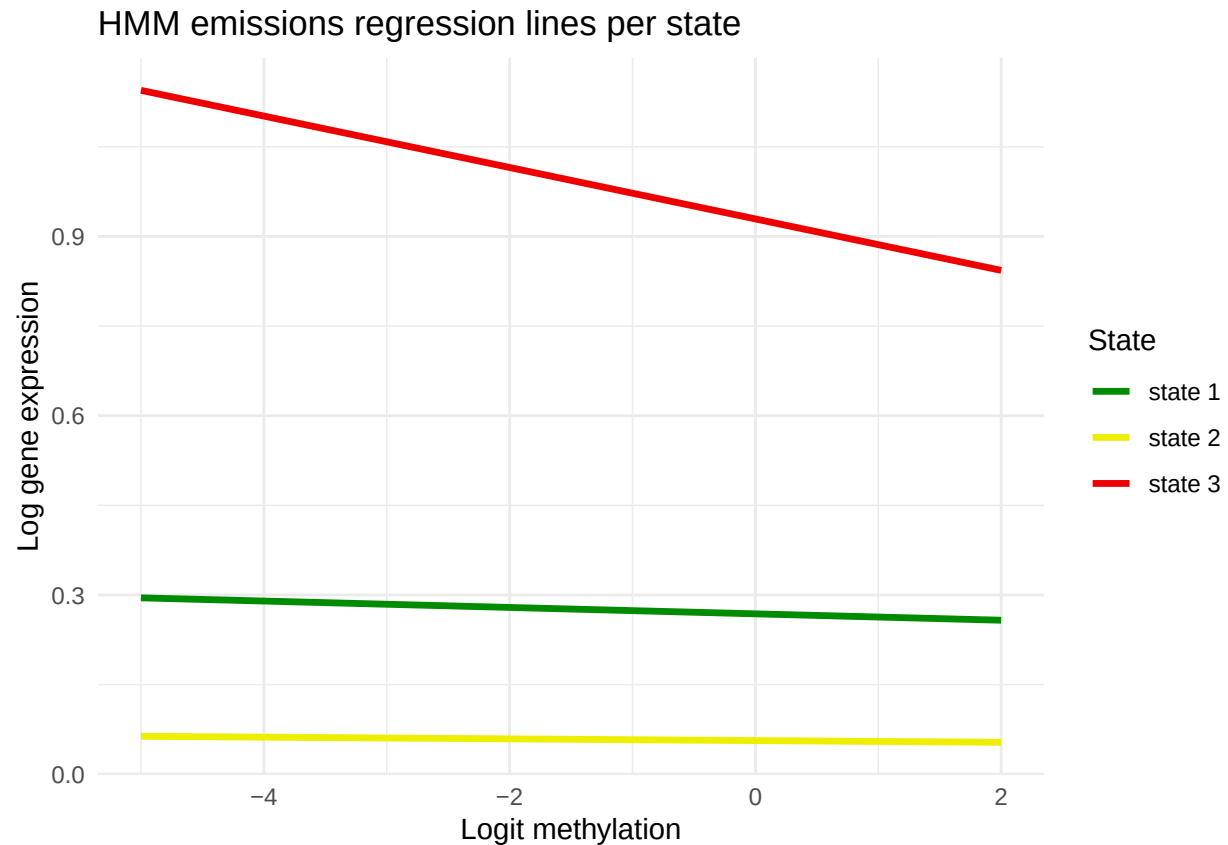
```
x_range <- seq(-5, 2, length.out = 100)
```

```
plot_emissions <- do.call(rbind, lapply(1:3, function(i) {
  data.frame(logit_methylation = x_range,
             log_expression = state_emissions$expression_intercept[i] +
               state_emissions$expression_slope[i] * x_range,
             state = state_emissions$state[i])
}))
```

```
# Claude-generated code for regression plot - we chose colors to match
# diagram in slides/report though!
```

```
ggplot(plot_emissions, aes(x = logit_methylation, y = log_expression, color = state)) + geom_line(size = 1)
```

```
## Warning: Using 'size' aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use 'linewidth' instead.
## This warning is displayed once per session.
## Call 'lifecycle::last_lifecycle_warnings()' to see where this warning was
## generated.
```



```
#####
# final r and r^2 values (averaged accordingly) #
#####

mean_lasso_r_squared <- mean(lasso_result$r_squared, na.rm = TRUE)
mean_hmm_r_squared <- mean(hmm_result$r_squared, na.rm = TRUE)

cat("R for simple correlation:",
    "\n\t all genes = ", cor_all,
    "\n\t expressed genes only = ", cor_expressed,
    "\nR^2 for LASSO = ", mean_lasso_r_squared,
    "\nR^2 for HMM = ", mean_hmm_r_squared)
```

```
## R for simple correlation:
##   all genes = -0.4454728
##   expressed genes only = -0.3696929
## R^2 for LASSO = 0.1453043
## R^2 for HMM = 0.633731
```

```
# for n genes = 500,
# cor = -0.433092
# cor -0.4313537
# lasso r_squared = 0.1398693
# hmm r_squared = 0.4878641
```

```
#for n genes = 1000,  
# cor = -0.433092  
# cor = -0.4313537  
# lasso r_squared = 0.1237019  
# hmm r_squared = 0.4643905
```